The Quantum Fourier Transform

Mathematics, Circuit Decomposition, and Implementation

Outline

1. Motivation and definition
2. From the classical DFT to the QFT
3. Derivation of the product form
4. Circuit decomposition with Hadamards and controlled phases
5. Worked 2- and 3-qubit examples
6. Approximate QFT and complexity
7. Connection to quantum phase estimation
8. Implementation logic and simulation remarks

Motivation

The Quantum Fourier Transform (QFT) is one of the central primitives in quantum computing.
It appears in:

- Shor's factoring algorithm,
- order finding,
- period finding,
- quantum phase estimation,
- several quantum simulation algorithms.

The QFT is the quantum analogue of the discrete Fourier transform, but it acts on the amplitudes of a quantum state.

Classical Discrete Fourier Transform

Given a vector $x = (x_0, x_1, \ldots, x_{N-1})$, the discrete Fourier transform is

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi ijk/N}, \qquad k = 0, \ldots, N-1.$$

Equivalently, the DFT matrix is

$$F_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}$$

with

$$\omega = e^{2\pi i/N}.$$

Definition of the QFT

For $N = 2^n$, the QFT is the unitary transformation

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle , \qquad x = 0, \ldots, N-1.$$

Thus the QFT maps a computational basis state $|x\rangle$ to a coherent superposition of all basis states with phases determined by x. Because the map is linear, for a general state

$$|\psi\rangle = \sum_{x=0}^{N-1} c_x |x\rangle ,$$

we have

$$\mathrm{QFT}_N |\psi\rangle = \sum_{x=0}^{N-1} c_x \, \mathrm{QFT}_N |x\rangle .$$

Binary Representation of the Input

Let

$$x = x_1 x_2 \dots x_n$$

be the binary representation of the integer x, meaning

$$x = \sum_{j=1}^{n} x_j 2^{n-j}, \qquad x_j \in \{0, 1\}.$$

We also write binary fractions as

$$0.x_j x_{j+1} \dots x_n = \frac{x_j}{2} + \frac{x_{j+1}}{2^2} + \dots + \frac{x_n}{2^{n-j+1}}.$$

This notation is crucial because the QFT phases factor naturally into such binary fractions.

Step-by-Step Derivation of the Product Form I

Start from

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle .$$

Write the output label k in binary:

$$k = k_1 k_2 \dots k_n, \qquad k = \sum_{m=1}^{n} k_m 2^{n-m}.$$

Then

$$\frac{xk}{2^n} = x \sum_{m=1}^{n} \frac{k_m}{2^m}.$$

Hence

$$e^{2\pi i x k/2^n} = \prod_{m=1}^{n} e^{2\pi i x k_m/2^m}.$$

Step-by-Step Derivation of the Product Form II

Because each $k_m \in \{0,1\}$, the sum over all k factorizes:

$$\mathrm{QFT}_N |x\rangle = \frac{1}{2^{n/2}} \bigotimes_{m=1}^{n} \left(|0\rangle + e^{2\pi i x/2^m} |1\rangle \right).$$

Now only the fractional part of $x/2^m$ matters in the phase, and that fractional part is exactly

$$0.x_{n-m+1}x_{n-m+2}\ldots x_n.$$

Thus

$$\mathrm{QFT}_N |x_1x_2\ldots x_n\rangle = \frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i\, 0.x_n} |1\rangle \right) \otimes \left(|0\rangle + e^{2\pi i\, 0.x_{n-1}x_n} |1\rangle \right) \otimes \cdots$$

$$\cdots \otimes \left(|0\rangle + e^{2\pi i\, 0.x_1x_2\ldots x_n} |1\rangle \right).$$

The output order is therefore reversed relative to the input qubit order.

Final Product Form

The standard product form is

$$\mathrm{QFT}_N \left|x_1 x_2 \ldots x_n\right\rangle = \frac{1}{2^{n/2}} \left(\left|0\right\rangle + e^{2\pi i\, 0.x_n} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_{n-1}x_n} \left|1\right\rangle\right) \otimes \cdots$$

$$\cdots \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_1 x_2 \ldots x_n} \left|1\right\rangle\right).$$

This factorization is the key reason the QFT can be implemented efficiently.
It shows that the QFT can be built from:

- Hadamard gates,
- controlled phase rotations,
- a final qubit reversal using SWAP gates.

Basic Gates Used in the QFT

The main one-qubit gate is the Hadamard

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

which creates equal superpositions.
The phase rotations are

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}.$$

Controlled-R_k gates apply R_k to a target qubit only if the control qubit is $|1\rangle$.

How the Circuit Arises

The first output qubit in the product form contains the phase

$$e^{2\pi i\, 0.x_1 x_2 \ldots x_n}.$$

This is produced by:

- a Hadamard on the first qubit, generating $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$,
- then a sequence of controlled phase gates from the later qubits, adding the binary-fraction phase.

The same pattern repeats recursively for the remaining qubits. Thus the QFT circuit is naturally built qubit by qubit.

Two-Qubit QFT: Mathematical Form

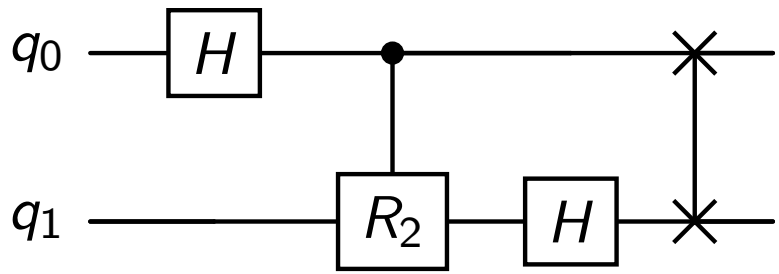
For $n = 2$, we have $N = 4$, and

$$\mathrm{QFT}_4 \left|x_1 x_2\right\rangle = \frac{1}{2}\left(\left|0\right\rangle + e^{2\pi i\, 0.x_2} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_1 x_2} \left|1\right\rangle\right).$$

This requires:

- Hadamard on the first qubit,
- controlled-R_2,
- Hadamard on the second qubit,
- then a SWAP to correct the reversed output order.

Two-Qubit QFT Circuit



Here:

- H creates the superposition,
- $R_2 = \mathrm{diag}(1, e^{i\pi/2})$,
- the SWAP reverses the qubit order.

Three-Qubit QFT: Mathematical Structure

For $n = 3$,

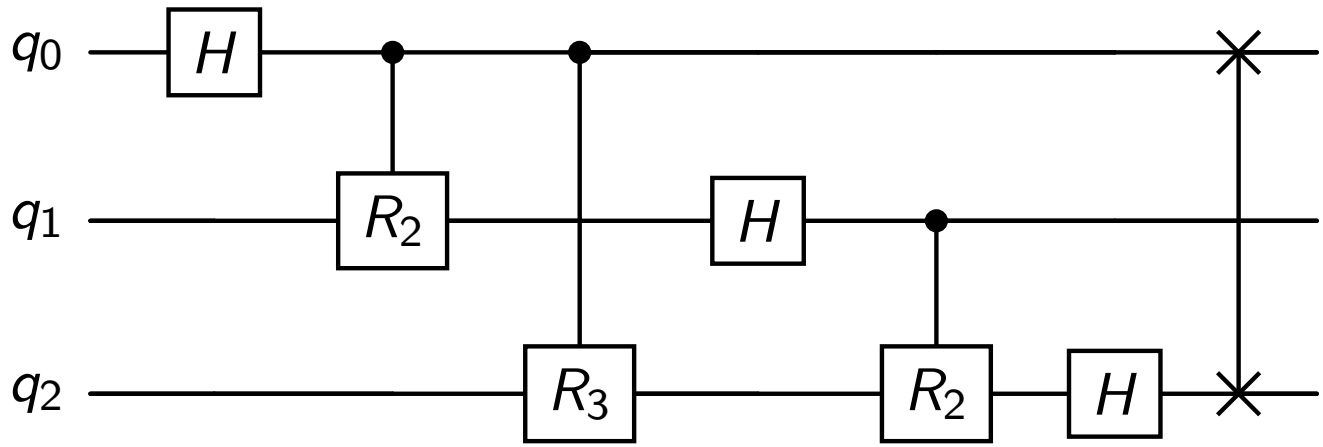
$$\mathrm{QFT}_8 \left|x_1 x_2 x_3\right\rangle = \frac{1}{\sqrt{8}} \left(\left|0\right\rangle + e^{2\pi i\, 0.x_3} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2\pi i\, 0.x_2 x_3} \left|1\right\rangle\right) \otimes \left(\left|0\right\rangle + e^{2}\right.$$

The first qubit therefore requires phase contributions corresponding to

$$\frac{x_2}{2} + \frac{x_3}{4},$$

and this is exactly what the controlled R_2 and R_3 gates generate.

Three-Qubit QFT Circuit



Conceptually:

1. H on qubit 0, then controlled R_2, R_3
2. H on qubit 1, then controlled R_2
3. H on qubit 2
4. final reversal of qubit order

General n-Qubit QFT Circuit

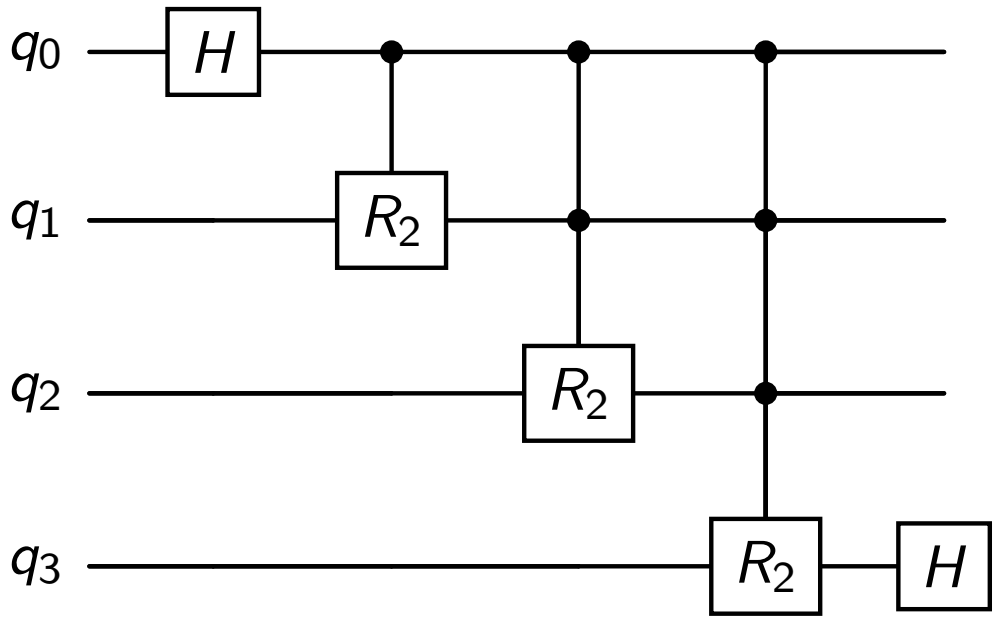
For qubit j, counting from the top:

1. Apply a Hadamard H
2. For each later qubit $k > j$, apply a controlled R_{k-j+1}
3. Continue recursively
4. At the end, reverse qubit order with SWAP gates

So the general structure is:

$H \rightarrow$ controlled phases $\rightarrow H \rightarrow$ controlled phases $\rightarrow \cdots \rightarrow$ SWAP networ

General Pattern in Quantikz



This is a schematic pattern rather than a fully expanded full-n circuit. The key point is the nested structure of Hadamards and controlled phase gates.

Worked 3-Qubit Phase Example I

Take the computational basis input

$$|x\rangle = |101\rangle\,.$$

This means

$$x_1 = 1, \qquad x_2 = 0, \qquad x_3 = 1, \qquad x = 5.$$

The product form gives

$$\mathrm{QFT}_8\,|101\rangle = \frac{1}{\sqrt{8}}\left(|0\rangle + e^{2\pi i\,0.1}\,|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i\,0.01}\,|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i\,0.1}\right.$$

Worked 3-Qubit Phase Example II

Now evaluate the binary fractions:

$$0.1 = \frac{1}{2}, \qquad 0.01 = \frac{1}{4}, \qquad 0.101 = \frac{1}{2} + \frac{1}{8} = \frac{5}{8}.$$

Hence

$$e^{2\pi i\, 0.1} = e^{i\pi} = -1,$$

$$e^{2\pi i\, 0.01} = e^{i\pi/2} = i,$$

$$e^{2\pi i\, 0.101} = e^{2\pi i(5/8)} = e^{5\pi i/4}.$$

Therefore

$$\mathrm{QFT}_8 \left|101\right\rangle = \frac{1}{\sqrt{8}}(\left|0\right\rangle - \left|1\right\rangle) \otimes (\left|0\right\rangle + i \left|1\right\rangle) \otimes (\left|0\right\rangle + e^{5\pi i/4} \left|1\right\rangle).$$

Worked 3-Qubit Phase Example III

This example shows exactly how the QFT encodes the binary digits of the input into relative phases.
This phase encoding is what makes the QFT so useful in:

- phase estimation,
- period finding,
- extracting eigenphases of unitaries.

In other words, the QFT converts arithmetic structure into measurable interference structure.

Inverse QFT

The inverse QFT is obtained by reversing the circuit and taking Hermitian conjugates:

- replace each R_k by $R_k^\dagger$,
- reverse the order of the gates,
- keep the same SWAP network.

Thus the inverse QFT is also efficient.
In practice, many algorithms use the inverse QFT rather than the forward QFT.

Complexity of the QFT

For n qubits:

- n Hadamard gates,
- $\frac{n(n-1)}{2}$ controlled phase gates,
- about $\lfloor n/2 \rfloor$ SWAP gates.

Therefore the total gate count is

$$O(n^2).$$

This is exponentially more efficient than applying the classical DFT matrix directly to a 2^n-dimensional vector.

Approximate QFT

Very small-angle phase gates contribute little.
Hence we may truncate the circuit by dropping controlled R_k gates for large k.
This gives the approximate QFT:

- fewer gates,
- shallower circuit,
- often still sufficient for practical algorithms.

With suitable truncation, the gate complexity can be reduced to approximately

$$O(n \log n).$$

Connection to Quantum Phase Estimation I

Suppose

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle$$

for some eigenstate $|\psi\rangle$.
Quantum phase estimation aims to determine the phase ϕ.
The algorithm uses:

- a control register prepared in superposition,
- controlled powers U^{2^j},
- an inverse QFT on the control register.

The inverse QFT converts the accumulated phase information into binary digits of ϕ.

Connection to Quantum Phase Estimation II

Schematically,

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i k \phi} |k\rangle$$

is transformed by the inverse QFT into a computational basis state approximating the binary expansion of ϕ.
Thus the QFT acts as a phase decoder.
This is why it is central to:

- Shor's algorithm,
- Hamiltonian eigenvalue estimation,
- quantum simulation,
- metrological phase extraction.

Implementation Logic in a Notebook or Simulator

A typical notebook implementation of the QFT proceeds as follows:

1. Choose the number of qubits n
2. Loop over qubits $j = 0, \ldots, n-1$
3. Apply H on qubit j
4. For each later qubit $k > j$, apply a controlled phase R_{k-j+1}
5. At the end, reverse the qubit order with SWAP gates

This is exactly the constructive logic derived from the product form.

Pseudocode for QFT Construction

```
for j = 0 to n − 1 :
    apply H on qubit j
    for k = j + 1 to n − 1 :
        apply controlled-R_{k−j+1} from k to j
for j = 0 to ⌊n/2⌋ − 1 :
    swap qubit j with qubit n − 1 − j
```

Depending on software conventions, the control and target order may appear reversed relative to the mathematical derivation, but the logical structure is the same.

Simulation Remarks

When simulating the QFT numerically, it is useful to compare:

- the circuit output state,
- the exact matrix formula

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle ,$$

- and the product-form prediction.

This provides a clean consistency check:

- amplitudes should have equal magnitude $1/\sqrt{N}$,
- phases should match the Fourier kernel,
- output qubit order must be checked carefully.

Common Source of Confusion: Bit Reversal

A frequent issue is the ordering of bits.
The product form naturally produces the qubits in reversed order:

$$(x_1 x_2 \dots x_n) \mapsto (\text{phase involving } x_n) \otimes \cdots \otimes (\text{phase involving } x_1 \dots x_n).$$

Thus:

- either include final SWAP gates,
- or accept the reversed output convention.

Many software libraries omit the final SWAPs and simply document the reversed ordering.

Why the QFT Matters Physically

The QFT is not just an abstract transform.
It provides a mechanism for:

- converting phase information into computational-basis information,
- exploiting interference to extract periodicity,
- connecting time evolution phases to measurable outputs.

In this sense, the QFT is a bridge between

$$\text{dynamical phase accumulation} \longrightarrow \text{algorithmic readout.}$$

Summary

The Quantum Fourier Transform:

- is the quantum analogue of the discrete Fourier transform,
- acts as
$$|x\rangle \mapsto \frac{1}{\sqrt{N}} \sum_k e^{2\pi i x k/N} |k\rangle ,$$
- factorizes into one-qubit phase states,
- is implemented using H, controlled R_k, and SWAP gates,
- has gate complexity $O(n^2)$,
- is central to phase estimation and many other algorithms.

Suggested Extensions

Natural next topics:

- inverse QFT in detail,
- quantum phase estimation,
- Shor's algorithm,
- approximate QFT and fault-tolerant implementations,
- explicit simulation in Qiskit or PennyLane.